

CS 284: Programming Assignment 1

Due: February 16, 2026

In Written Homework 1, you saw the notion of a `LinkedList`: A data structure implementing a `List` by storing the elements across several arrays, connected in series the way a `LinkedList` is.

In this Programming Assignment, you will now be implementing a full `LinkedList` in Java. While your end goal is to just implement the methods we have provided, we provide more detailed explanations, as well as a suggested order for handling them, in the document below.

Background

Let us begin by recalling what a `LinkedList` is.

Formally, a `LinkedList` is made up of a series of `ArrayNodes`, each of which has an `array` field and a `next` pointer. Each `ArrayNode` stores an array in its `array` field, and the `next` pointers point to the next `ArrayNode` in sequence. The data in the list is then spread out over these arrays, in order.

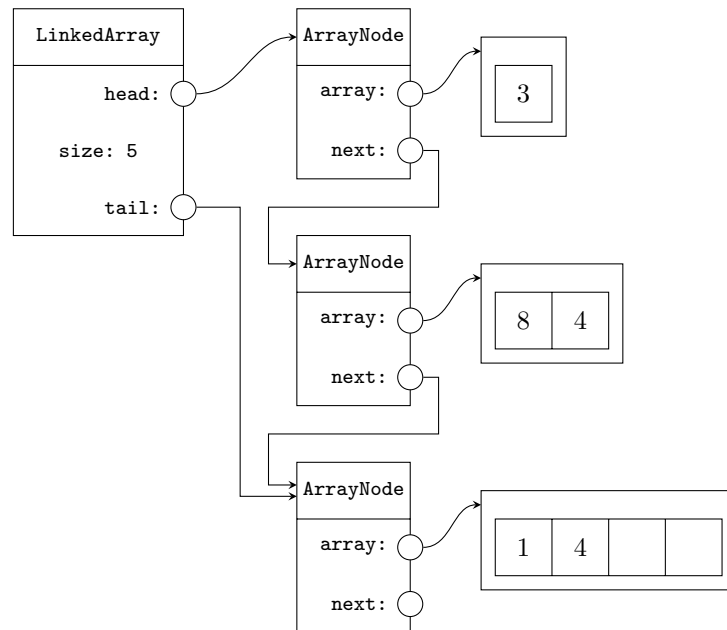
To avoid having to create a lot of different nodes, we implement the following rule: Each `ArrayNode` stores an array twice as large as the previous, with the lead node storing an array of size 1. In other words, the first `ArrayNode` has an array of size 1, the next has an array of size 2, then size 4, and so on.

Note that the remaining cells in the arrays are not used.

The `LinkedList` itself stores three values:

- `head`: A pointer to the first `ArrayNode` in the chain (`null` if the `LinkedList` is empty).
- `tail`: A pointer to the last `ArrayNode` in the chain (`null` if the `LinkedList` is empty).
- `size`: The number of elements in the list.

For example, the following is the general structure for a `LinkedList` representing the List `[3,8,4,1,4]`:



A Note on Access Control

Note that traditionally, our `LinkedListArray` would store its nodes as a `private` field, in order to separate the specification from the implementation.

However, we need to be able to access your `LinkedListArray` in order to test it and grade it. As such, we require that the fields we provide you must be marked `public`, so that our grading scripts can access them.

You may, however, mark any helper functions you create as `private`.

A Note on Starter Code

In order to be able to test your code, we need to ensure that we can create `ArrayNodes` within our test script. Thus, we require that you do not edit the `ArrayNode` class or its contents.

Task 1: General Implementation

Your first task is to implement the basic `LinkedListArray` API. In particular, your `LinkedListArray` should implement the following methods:

default Constructor: Your `LinkedListArray` should have a default zero-argument constructor. In other words, I should be able to call `new LinkedListArray()` to create a new (empty) `LinkedListArray`.

get: `get(idx)` should return the value in the `List` at index `idx`.

Example: Given a list containing `[3,8,4,1,4]`, `get(2)` should return `4`.

Precondition: For a list of length ℓ , the value `idx` must fall in the range $0 \leq \text{idx} \leq \ell - 1$.

set: `set(idx,value)` should update the value at index `idx` of the `List` to be `value`.

Example: given a list containing `[3,8,4,1,4]`, `set(3,5)` should update the list to be `[3,8,4,5,4]`.

Precondition: For a list of length ℓ , the value `idx` must fall in the range $0 \leq \text{idx} \leq \ell - 1$.

append: `append(value)` should add the given value to the end of your `List`.

Example: Given a list containing `[3,8,4,5,4]`, `append(7)` should update the list to be `[3,8,4,5,4,7]`.

pop: `pop()` should remove the last value in the list and return it.

Example: Given a list containing `[3,8,4,5,4,7]`, `pop()` should update the list to be `[3,8,4,5,4]` and return `7` (the removed element).

size: `size()` should return the number of elements in the list.

Example: Given a list containing `[3,8,4,5,4,7]`, `size()` should return `6`.

Precondition: The list must not be empty.

insert: (*OPTIONAL*) `insert(idx,value)` should insert `value` into the list such that it ends up at index `idx`, with values at or after `idx` being shifted right.

Example: Given a list containing `[3,8,4,5,7,4]`, `insert(3,6)` should update the list to be `[3,8,4,6,5,7,4]`.

Precondition: For a list of length ℓ , the value `idx` must fall in the range $0 \leq \text{idx} \leq \ell$. (Note carefully the end of this range!)

remove: (*OPTIONAL*) `remove(idx)` should remove the value at the index `idx` from the list. This should cause the list to shrink, with elements after `idx` moving up.

Example: Given a list containing `[3,8,4,6,5,7,4]`, `remove(1)` should return `[3,4,6,5,7,4]`.

Precondition: For a list of length ℓ , the value `idx` must fall in the range $0 \leq \text{idx} \leq \ell - 1$. The list must not be empty.

toString: (*OPTIONAL*) `toString()` should return a `String` representation of the `List` it is called on.

parametrized Constructor: (*OPTIONAL*) For testing purposes, you may wish to create an additional constructor for `LinkedListArray`, such that you can create a `LinkedListArray` with some initial data. However, note that you must still keep the default parameterless constructor.

Task 2: Input Validation

Note that a lot of the above methods have preconditions: Conditions that must be true of the list or the method arguments in order for the behaviour of the method to be well-defined. For example, you clearly cannot get the element at index 4 in a 3-element list.

Your next task is to make sure your methods have input validation. For each of your methods, add code to verify that the preconditions hold before they are run. Should a precondition be violated, your code should throw a `RuntimeException` with an appropriate message.

Suggested Implementation Order

Below, we give a suggested approach for how to tackle the above tasks, as well as a few hints on how to do it.

- Start with the constructor. How should the fields be instantiated so that the initial `LinkedListArray` is empty?
- Implement `get` and `set` next.
For this step, you may find helpful to write a shared helper function, as you will find that `get` and `set` have a lot of shared functionality.
- Implement the general case of `append`.
In other words, implement `append` for when you can just write to a cell in an existing `ArrayNode`, rather than needing to extend the chain by creating a new `ArrayNode`.
- Implement the full version of `append`, by handling the case where a new `ArrayNode` has to be created.
- Implement the general case of `pop`.
In other words, implement `pop` where the structure of the `ArrayNodes` doesn't have to change.
- Implement the full version of `pop`.
- Modify your previous methods to maintain the `size` field correctly.
- Implement `size`.
- Add input validation to your methods.
- (*Optional*) If you have time and interest, try implementing the remaining methods mentioned above.

Remember to test as you go, making sure that your `LinkedListArray` behaves as anticipated.

Additionally, remember that you are always allowed to write helper functions, which might be particularly useful when certain operations are repeated often.

Submission Instructions

For this homework, you must submit the following files, with exactly the following names:

- `LinkedListArray.java`

Note that `LinkedListArray` must be located in the `pa1` package, or we will be unable to compile it otherwise.

Remember to put your name and the Stevens Honor Pledge on every file you submit! Failure to put the pledge can result in marks being deducted, with repeat penalties potentially getting stricter between assignments.

Submissions must be handled through Gradescope (accessible through the homework page on Canvas). Grades will be released through Gradescope once the homework is graded.

Rubric

Category	Description	Points
<code>get</code>	Correct implementation of <code>get</code>	10
<code>set</code>	Correct implementation of <code>set</code>	10
<code>insert</code>	Correct implementation of <code>insert</code>	20
<code>remove</code>	Correct implementation of <code>remove</code>	20
<code>size</code>	Correct implementation of <code>size</code>	20
Input Validation	Correctly handling invalid inputs in the above methods	20
Total		100